Artificial Intelligence



Problem Solving by Uniformed Search

Conducted by:

Dr. Fazlul Hasan Siddiqui

(fazlul.siddiqui@duet.ac.bd)

Credit: MIT, USA

Problem Solving and Search

Problem Solving

- Agent knows world dynamics [learning]
- World state is finite, small enough to enumerate [logic]
- World is deterministic [uncertainty]
- Utility for a sequence of states is a sum over path
- Agent knows current state [logic, uncertainty]

Few real problems are like this, but this may be a useful abstraction of a real problem

Example: Route Planning in a Map

A map is a graph where nodes are cities and links are roads. This is an abstraction of the real world.

- Map gives world dynamics: starting at city X on the map and taking some road gets to you to city Y.
- World (set of cities) is finite and enumerable.
- World is deterministic: taking a given road from a given city leads to only one possible destination.
- Utility for a sequence of states is usually either total distance traveled on the path or total time for the path.
- We assume current state is known

Formal Definition

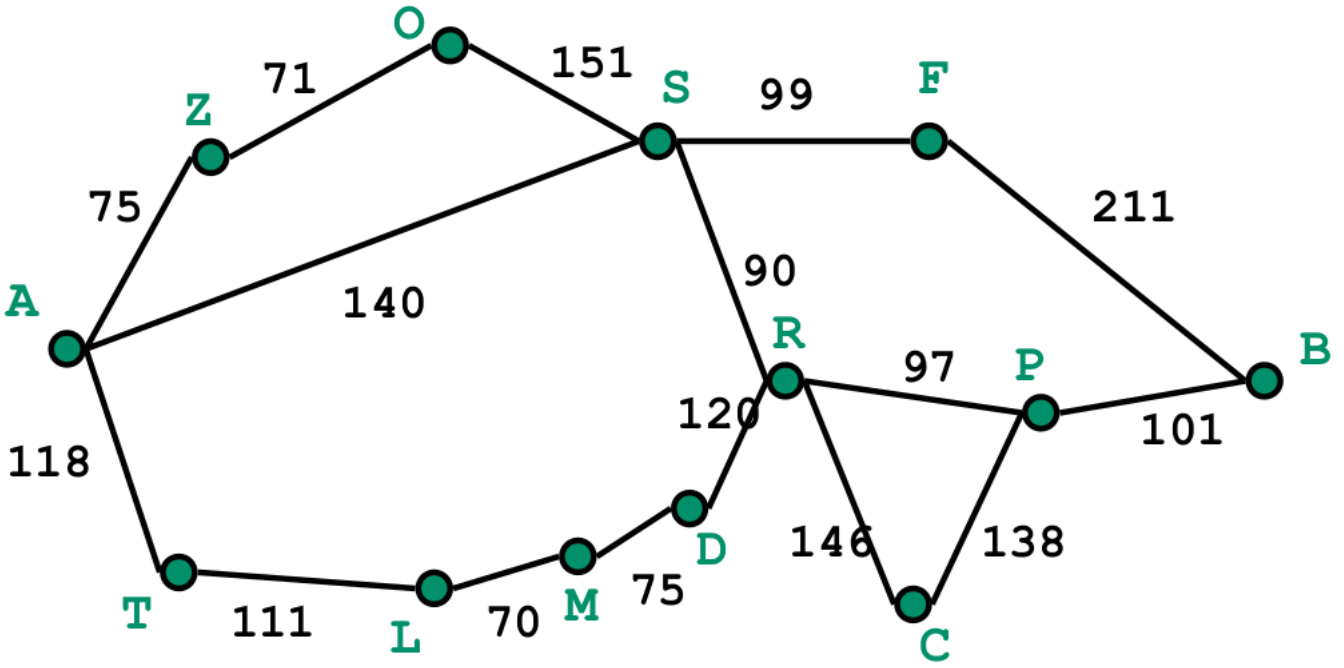
Problem:

- Set of states: S
- Initial state
- Operators (actions): $S \rightarrow S$
- Goal test: $S \rightarrow \{ t, f \}$
- Path cost: $(S, O)^* \rightarrow \mathbb{R}$
  - Sum of costs: $\sum c(S,O)$

Criteria for algorithms:

- Computation time/space
- Solution quality

Route Finding



Romania Map


HUNGARY
MOLDOVA
UKRAINE
Serbia
BULGARIA
THE FORMER YUGOSLAV REPUBLIC OF MACEDONIA
Belgrade
Bucharest
Chisinău
Constanța
Black Sea
BOTOȘANI
IAȘI
NEAMȚ
BACĂU
VASLUI
GALAȚI
VRANCEA
BUZĂU
BRĂILA
TULCEA
CONSTANȚA
CĂLĂRAȘI
IALOMIȚA
PRAHOVA
DÂMBOVIȚA
ARGEȘ
OLT
DOLJ
MEHEDINȚI
GORJ
CARAȘ SEVERIN
TIMIȘ
ARAD
HUNEDOARA
ALBA
SIBIU
BRAȘOV
COVASNA
HARGHITA
MUREȘ
CLUJ
BIHOR
SATU MARE
MARAMUREȘ
SUCEAVA
Romania
International boundary
County (județ) boundary
National capital
County (județ) center
Railroad
Expressway
Road
Bucuresti is a municipality (municipiu).
0 25 50 75 100 Kilometers
0 25 50 75 100 Miles
Lambert Conformal Conic Projection, SP 40N/50N

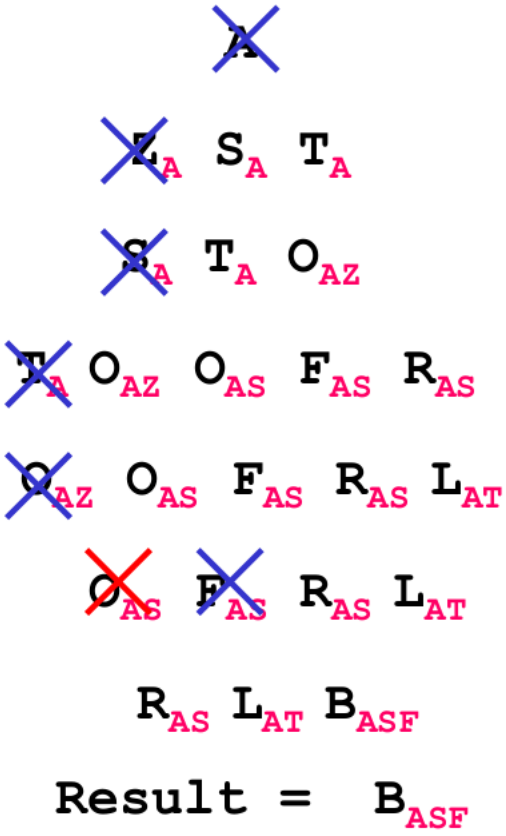
Search

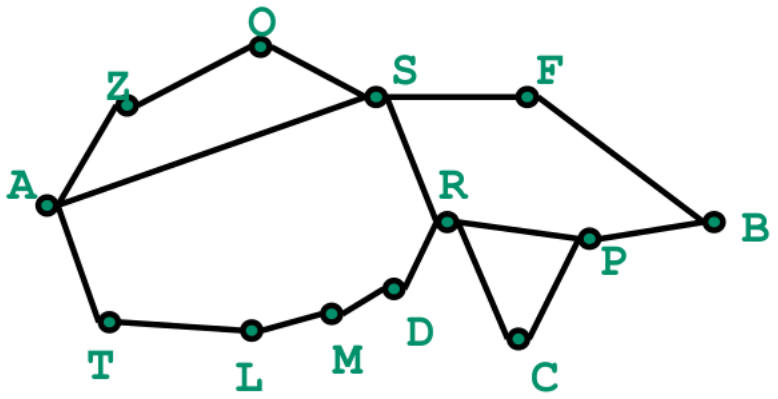
- Put start state in the agenda
- Loop
  - Get a state from the agenda
    - If goal, then return
    - Expand state (put children in agenda)

Which state is chosen from the agenda defines the type of search and may have huge impact on effectiveness.

Breadth-First Search

- Treat agenda as a queue (get least recently added node)
  - Expansion: put children at end of queue
  - Get new nodes from the front of queue





Avoiding Loops

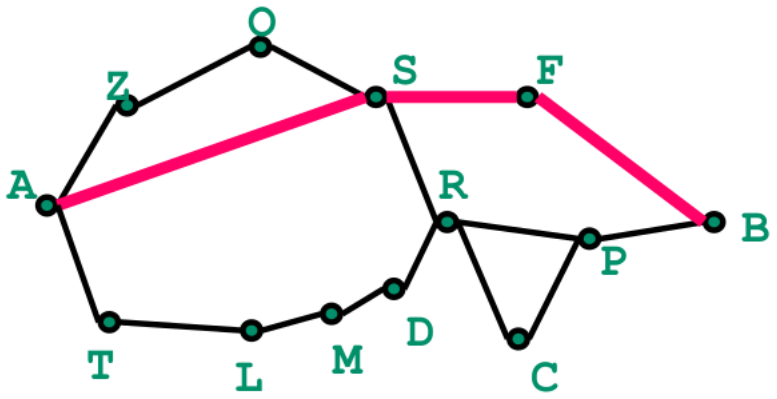
- Method 1:
  - Don’t add a node to the agenda if it’s already in the agenda
  - Causes problems when there are multiple paths to a node and we want to be sure to get the shortest
- Method 2:
  - Don’t expand a node (or add it to the agenda) if it has already been expanded.
  - We’ll adopt this one for all of our searches

Properties of BFS

Guaranteed to return shortest path (measured in number of arcs) from start to goal.

Let

- b = branching factor
- m = maximum depth
- d = goal depth



- $O(b^d)$ time
- $O(b^d)$ space

Properties of BFS

Depth	Nodes	Time	Memory
2	110	.11 milliseconds	107 kilobytes
4	11,110	11 milliseconds	10.6 megabytes
6	10^6	1.1 seconds	1 gigabyte
8	10^8	2 minutes	103 gigabytes
10	10^{10}	3 hours	10 terabytes
12	10^{12}	13 days	1 petabyte
14	10^{14}	3.5 years	99 petabytes
16	10^{16}	350 years	10 exabytes

Figure 3.13 Time and memory requirements for breadth-first search. The numbers shown assume branching factor $b = 10$; 1 million nodes/second; 1000 bytes/node.

Depth-First Search

- Treat agenda as a stack (get most recently added node)
  - Expansion: put children at top of stack
  - Get new nodes from top of stack

~~A~~

~~Z~~$_A$ S_A T_A

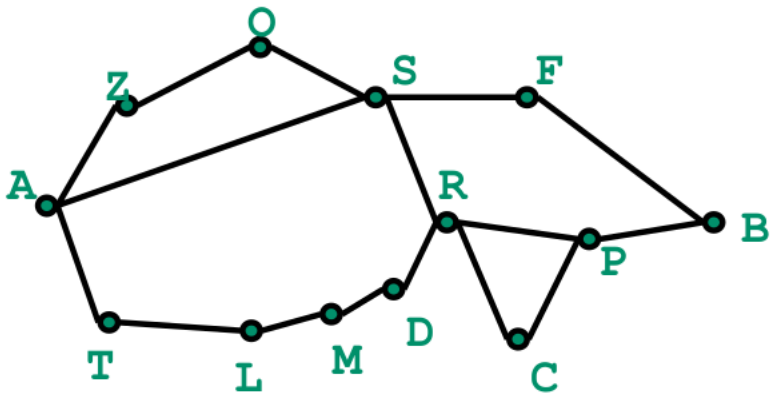
~~O~~$_{AZ}$ S_A T_A

~~S~~$_{AZO}$ S_A T_A

~~F~~$_{AZOS}$ R_{AZOS} S_A T_A

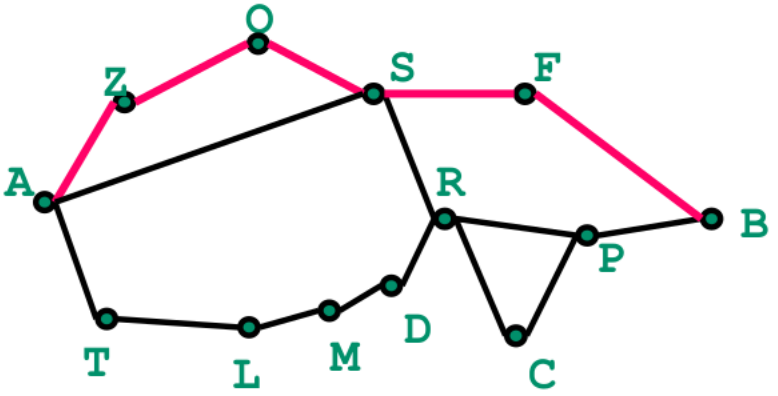
~~B~~$_{AZOSF}$ R_{AZOS} S_A T_A

Result = B_{AZOSF}



Properties of DFS

Sub-optimal answer:



Let

- b = branching factor
- m = maximum depth
- d = goal depth

- $O(b^m)$ time
- O(mb) space

Properties of BFS

Depth	Nodes	Time	Memory
2	110	.11 milliseconds	107 kilobytes
4	11,110	11 milliseconds	10.6 megabytes
6	10^6	1.1 seconds	1 gigabyte
8	10^8	2 minutes	103 gigabytes
10	10^{10}	3 hours	10 terabytes
12	10^{12}	13 days	1 petabyte
14	10^{14}	3.5 years	99 petabytes
16	10^{16}	350 years	10 exabytes

Figure 3.13 Time and memory requirements for breadth-first search. The numbers shown assume branching factor $b = 10$; 1 million nodes/second; 1000 bytes/node.

16 10^{16} 350 years

DFS: 156 kilobytes

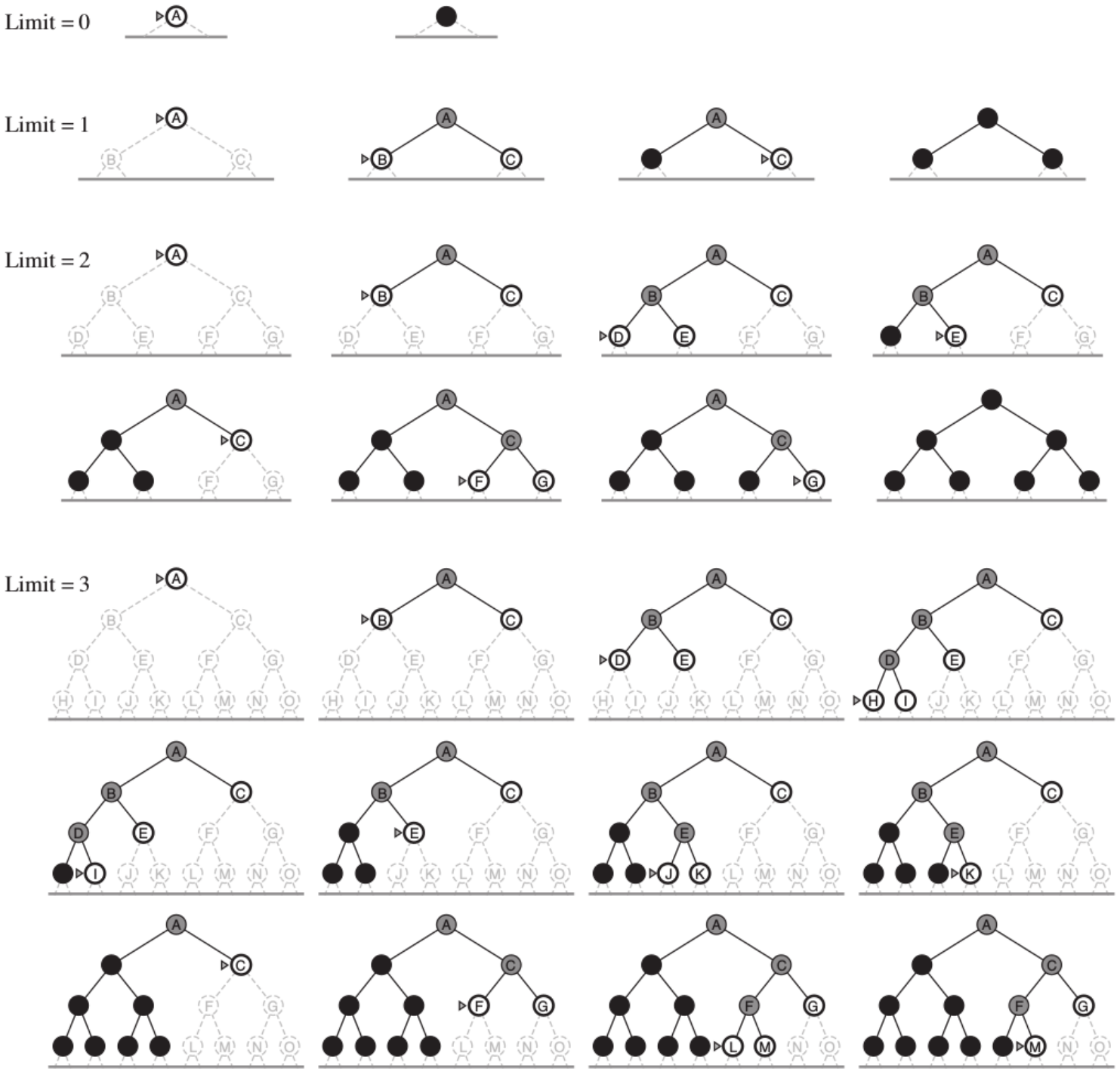
Backtracking Search

- A variant of depth-first search called backtracking search uses still less memory.
- In backtracking, only one successor is generated at a time rather than all successors
- Each partially expanded node remembers which successor to generate next.
- In this way, only O(m) memory is needed rather than O(bm).

Depth-limited search

- That is, nodes at depth l are treated as if they have no successors.
- Depth-first search can be viewed as a special case of depth-limited search with $l = \infty$
- Its time complexity is $O(b^l)$ and its space complexity is $O(bl)$.
- can be viewed as a special case of depth-limited search with $l = \infty$.
- Sometimes, depth limits can be based on knowledge of the problem. For example, on the map of Romania there are 20 cities. Therefore, we know that if there is a solution, it must be of length 19 at the longest, so $l = 19$ is a possible choice.
- But in fact if we studied the map carefully, we would discover that any city can be reached from any other city in at most 9 steps. This number, known as the diameter of the state space, gives us a better depth limit, which leads to a more efficient depth-limited search.

Iterative Deepening



Iterative Deepening

- DFS is efficient in space, but has no path-length guarantee
- BFS finds min-step path but requires exponential space
- Iterative deepening: Perform a sequence of DFS searches with increasing depth-cutoff until goal is found.

DFS cutoff depth	Space	Time
1	O(b)	O(b)
2	O(2b)	$O(b^2)$
3	O(3b)	$O(b^3)$
4	O(4b)	$O(b^4)$
…	…	…
d	O(db)	$O(b^d)$
Total	Max = O(db)	Sum = $O(b^{d+1})$

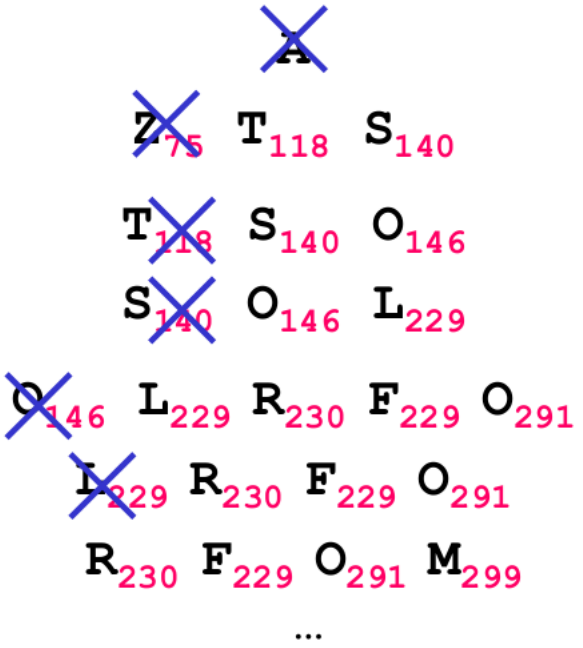
$$N(\mathrm{IDS}) = 50 + 400 + 3,000 + 20,000 + 100,000 = 123,450$$
$$N(\mathrm{BFS}) = 10 + 100 + 1,000 + 10,000 + 100,000 = 111,110\,.$$

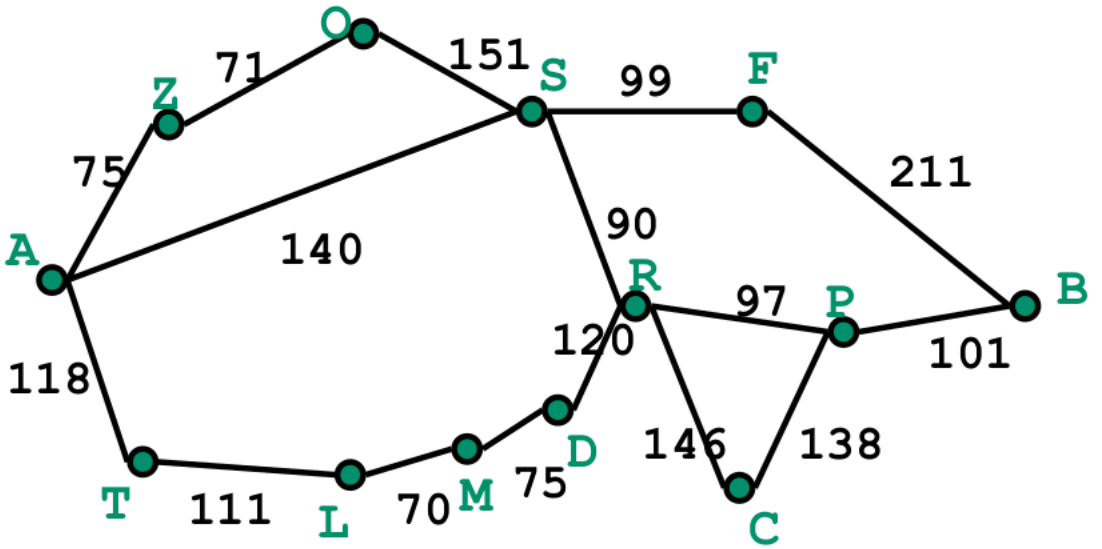
In general, iterative deepening is the preferred uninformed search method when the **search space is large and the depth of the solution is not known**.

Uniform Cost Search

- Breadth-first and Iterative-Deepening find path with fewest steps (hops).
- If steps have unequal cost, this is not interesting.
- How can we find the shortest path (measured by sum of distances along path)?
- Uniform Cost Search:
  - Nodes in agenda keep track of total path length from start to that node
  - Agenda kept in priority queue ordered by path length
  - Get shortest path in queue
- Explores paths in contours of total path length; finds optimal path.

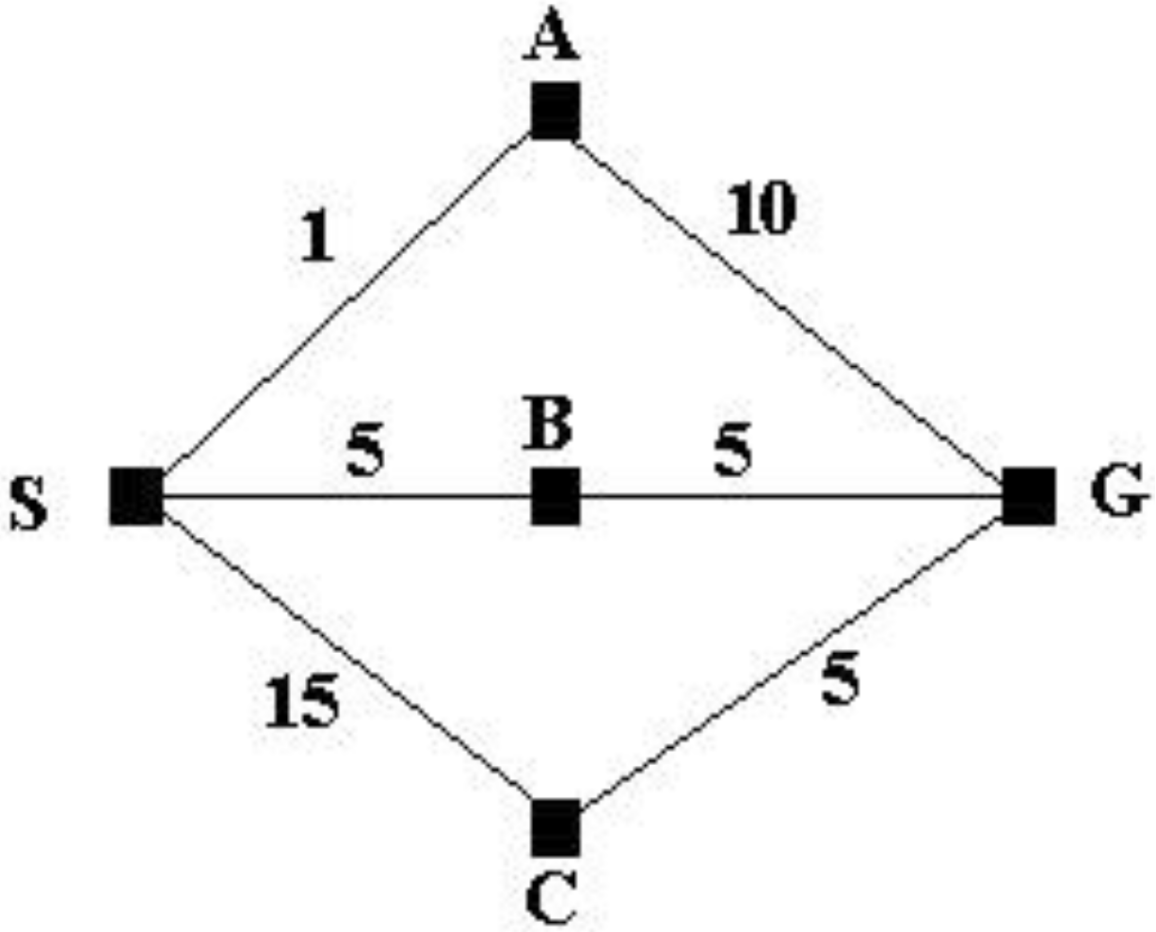
Uniform Cost Search





- We examine a node to see if it is the goal only when we take it off the agenda, not when we put it in.
- The algorithm is optimal only when the costs are non-negative.
- Time cost is $O(b^m)$ [not $O(b^d)$ as in Russell&Norvig]
- Space cost could also be $O(b^m)$, but is probably more like $O(b^d)$ in most cases.

Uniform Cost Search



Bidirectional Search

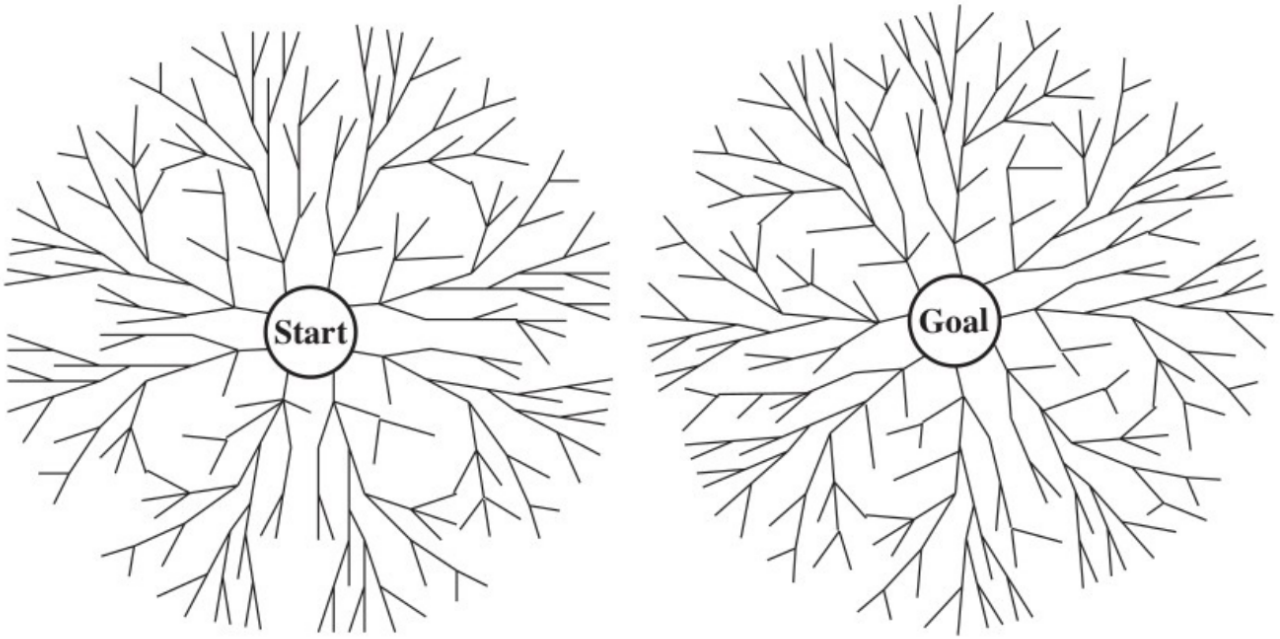


Figure 3.20 A schematic view of a bidirectional search that is about to succeed when a branch from the start node meets a branch from the goal node.

- The motivation is that b^(d/2) + b^(d/2) is much less than b^d
- For **b = 10**, this means a total of **2,220** node generations, compared with **1,111,110** for a standard breadth-first search.

Comparing uninformed search strategies

Criterion	Breadth-First	Uniform-Cost	Depth-First	Depth-Limited	Iterative Deepening	Bidirectional (if applicable)
Complete?	Yes^{a}	Yes^{a,b}	No	No	Yes^{a}	Yes^{a,d}
Time	$O(b^d)$	$O(b^{1+\lfloor C^*/\epsilon \rfloor})$	$O(b^m)$	$O(b^\ell)$	$O(b^d)$	$O(b^{d/2})$
Space	$O(b^d)$	$O(b^{1+\lfloor C^*/\epsilon \rfloor})$	$O(bm)$	$O(b\ell)$	$O(bd)$	$O(b^{d/2})$
Optimal?	Yes^{c}	Yes	No	No	Yes^{c}	Yes^{c,d}

Figure 3.21 Evaluation of tree-search strategies. b is the branching factor; d is the depth of the shallowest solution; m is the maximum depth of the search tree; l is the depth limit. Superscript caveats are as follows: ^{a} complete if b is finite; ^{b} complete if step costs $\geq \epsilon$ for positive ϵ; ^{c} optimal if step costs are all identical; ^{d} if both directions use breadth-first search.